```
0000 SEPARATE(¬IDL )¶--|----------------------------------------------------------------------------------------------------¶--|¬¬LALR_GRMR¶PROCEDURE¬LALR_GRM
     R¬( NOM_TEXTE :STRING¬) IS¶¶  TYPE STBL_TYPE¬IS RECORD¶¬¬  CHANGED, CLOSURE¬: BOOLEAN¬:= FALSE;¶¬¬  STATE¬¬: TREE¬:= TREE_VOID;¶¬¬END RECORD;¶  STBL¬:
      ARRAY( 1 .. 1100 ) OF STBL_TYPE;¶¶  TYPE RTBL_TYPE¬IS RECORD¶¬¬  STATE_NBR¬: INTEGER¬:= 0;¶¬¬  REQ_CHECK¬: BOOLEAN;¶¬¬  FOLLOW¬¬: TREE;¶¬¬END RECORD;
     ¶  RTBL¬: ARRAY( 1 .. 400 )¬OF RTBL_TYPE;¶¶  MORE_PASSES¬¬: BOOLEAN;¶  MORE_CLOSURE_PASSES¬: BOOLEAN;¶  GR_STATE_SEQ¬¬: SEQ_TYPE;¶¶  --|--------------
     ----------------------------------------------------------------------------------------¶  --|¬¬FUNCTION GET_RULE_NBR¶  FUNCTION GET_RULE_NBR ( SYL_LIST :SEQ
     _TYPE ) RETURN INTEGER IS¶    SYL¬: TREE;¶    RULE¬: TREE;¶  BEGIN¶    IF IS_EMPTY( SYL_LIST ) THEN¶      RETURN 0;¶    END IF;¶    SYL := HEAD( SYL_L
     IST );¶    IF SYL.TY /= DN_NONTERMINAL THEN¶      RETURN 0;¶    END IF;¶    RULE := D( XD_RULE, SYL );¶    IF RULE.TY = DN_VOID THEN¶      RETURN 0;¶
        END IF;¶    RETURN DI( XD_RULE_NBR, D( XD_RULEINFO, RULE ) );¶  END;¶  --|--------------------------------------------------------------------------------
     ----------------------¶  --|¬¬PROCEDURE¬INITIALIZE¶  PROCEDURE INITIALIZE IS¶    STATE_SEQ¬: SEQ_TYPE¬:= GR_STATE_SEQ;¶    STATE¬¬: TREE;¶    STATE_NB
     R¬: INTEGER;¶    ITEM_SEQ¬: SEQ_TYPE;¶    ITEM¬¬: TREE;¶  BEGIN¶    WHILE¬NOT IS_EMPTY( STATE_SEQ) LOOP¶      POP( STATE_SEQ, STATE );¶      STATE_NBR
0010  := DI( XD_STATE_NBR, STATE );¶      PUT( "INIT" );¶      INT_IO.PUT( STATE_NBR );¶      NEW_LINE;¶      STBL( STATE_NBR ).STATE¬:= STATE;¶¶      ITEM
     _SEQ := LIST( STATE);¶      WHILE NOT IS_EMPTY( ITEM_SEQ ) LOOP¶        POP( ITEM_SEQ, ITEM );¶¶INIT_MAKE_RTBL_ONE_ITEM:¶        DECLARE¶¬FOLLOW¬: TRE
     E;¶        BEGIN¶¬IF DI ( XD_SYL_NBR,¬ITEM ) = 0 THEN¬¬¬-- CLOSURE ITEM¶¬  STBL( STATE_NBR ).CLOSURE := TRUE;¶¶¬  DECLARE¶¬    ALTERNATIVE¬: TREE¬¬:=
     D( XD_ALTERNATIVE, ITEM );¶¬    RULE¬¬: TREE¬¬:= D( XD_RULE, ALTERNATIVE );¶¬    RULE_INFO¬: TREE¬¬:= D( XD_RULEINFO, RULE );¶¬    RULE_NBR¬: INTEGER¬
     ¬:= DI( XD_RULE_NBR,¬RULE_INFO¬);¶¬    RTBL_I¬: RTBL_TYPE¬RENAMES RTBL( RULE_NBR );¶¬  BEGIN¶¬    IF RTBL_I.STATE_NBR /= STATE_NBR THEN¬¬¬-- HAVE NOT A
     LREADY¬SEEN A CLOSURE ITEM¬FOR THIS RULE¶¬      RTBL_I.STATE_NBR := STATE_NBR;¶¬      RTBL_I.FOLLOW¬:= MAKE( DN_TERMINAL_S );¶¬      LIST( RTBL_I.FOLL
     OW, (TREE_NIL,TREE_NIL) );¶¬    END IF;¶¬    FOLLOW := RTBL_I.FOLLOW;¶¬  END;¶¶¬ELSE¬¬¬¬-- BASIS ITEM¶¬  FOLLOW := MAKE( DN_TERMINAL_S );¶¬  LIST( FOL
     LOW, (TREE_NIL,TREE_NIL) );¶¬END IF;¶¬D( XD_FOLLOW, ITEM,¬FOLLOW );¶        END INIT_MAKE_RTBL_ONE_ITEM;¶¶      END¬LOOP;¶¶      IF STBL( STATE_NBR ).
     CLOSURE THEN¶        ITEM_SEQ :=¬LIST( STATE );¶        WHILE NOT IS_EMPTY( ITEM_SEQ ) LOOP¶¬POP( ITEM_SEQ, ITEM¬);¶¶INIT_CLOSE_RTBL_ONE_ITEM:¶¬DECLAR
     E¶¬  SYL_LIST¬: SEQ_TYPE¬:= LIST( ITEM );¶¬  RULE_NBR¬: INTEGER;¶¬  SYL¬¬: TREE;¶¬  RULE¬¬: TREE;¶¬  FOLLOW¬¬: TREE;¶¬  FOLLOW_SEQ¬: SEQ_TYPE;¶¬  FOLL
0020 OW_SAVE¬: SEQ_TYPE;¶¬BEGIN¶¬  RULE_NBR := GET_RULE_NBR( SYL_LIST );¶¶¬  IF RULE_NBR = 0 THEN¶¬    GOTO FIN;¶¬  END IF;¶¶¬  SYL_LIST := TAIL(¬SYL_LIST
     );¶¬  IF IS_EMPTY( SYL_LIST ) THEN¶¬    GOTO FIN;¶¬  END IF;¶¶¬  IF RTBL( RULE_NBR¬).STATE_NBR /= STATE_NBR THEN¶¬    PUT( "*** RULE TABLE INCORRECT."
     );¶¬    INT_IO.PUT( RULE_NBR );¶¬    INT_IO.PUT( RTBL( RULE_NBR ).STATE_NBR );¶¬    INT_IO.PUT( STATE_NBR );¶¬    NEW_LINE;¶¬    GOTO FIN;¶¬  END IF;
     ¶¶¬  FOLLOW := RTBL( RULE_NBR ).FOLLOW;¶¬  FOLLOW_SEQ := LIST( FOLLOW );¶¬  FOLLOW_SAVE := FOLLOW_SEQ;¶¶¬  LOOP¶¬    POP( SYL_LIST, SYL );¶¬    IF SYL
     .TY = DN_TERMINAL THEN¶¬      FOLLOW_SEQ :=¬TERM_LIST.UNION( FOLLOW_SEQ, SYL );¶¬      EXIT;¶¬    ELSE¶¬      RULE := D( XD_RULE, SYL¬);¶¬      IF RUL
     E.TY = DN_VOID THEN¶¬      EXIT;¶¬      END¬IF;¶¬      FOLLOW_SEQ :=¬TERM_LIST.UNION( FOLLOW_SEQ, LIST( D( XD_RULEINFO, RULE ) )¬);¶¬      IF NOT DB
     ( XD_IS_NULLABLE, RULE )¬THEN¶¬      EXIT;¶¬      END¬IF;¶¬    END IF;¶¬    EXIT WHEN IS_EMPTY( SYL_LIST );¶¬  END LOOP;¶¶¬  IF NOT TERM_LIST.SAME(
     FOLLOW_SEQ, FOLLOW_SAVE ) THEN¶¬    LIST(¬FOLLOW, FOLLOW_SEQ );¶¬    STBL(¬STATE_NBR¬).CHANGED¬:= TRUE;¶¬  END IF;¶¬END INIT_CLOSE_RTBL_ONE_ITEM;¶<<FI
     N>>¶¬NULL;¶      END LOOP;¶    END¬IF;¶    END LOOP;¶  END INITIALIZE;¶  --|--------------------------------------------------------------------------------
     ----------------------¶  --|¬¬PROCEDURE¬TRANS_CLOSE¶  PROCEDURE TRANS_CLOSE IS¶    STATE¬¬: TREE;¶    RULE¬¬: TREE;¶    RULE_NBR¬: INTEGER;¶    ITE
0030 M¬¬: TREE;¶    ITEM_SEQ¬: SEQ_TYPE;¶    ITEM_SUBSEQ¬: SEQ_TYPE;¶¶    --|--------------------------------------------------------------------------------
     ---------------¶    --|¬¬PROCEDURE¬TRANS_CLOSE_CLOSURE_ONE_ITEM¶    PROCEDURE TRANS_CLOSE_CLOSURE_ONE_ITEM ( STATE_NBR :INTEGER; ITEM¬:TREE ) IS¶
      SYL_LIST¬: SEQ_TYPE¬:= LIST( ITEM);¶      RULE_NBR¬: INTEGER;¶      SYL¬¬: TREE;¶      RULE¬: TREE;¶      FOLLOW¬: TREE;¶      FOLLOW_SEQ¬: SEQ_TYPE;
     ¶      FOLLOW_SAVE¬: SEQ_TYPE;¶    BEGIN¶      RULE_NBR := GET_RULE_NBR( SYL_LIST );¶      IF RULE_NBR =¬0 THEN¶        RETURN;¶      END¬IF;¶      IF
      RTBL( RULE_NBR ).STATE_NBR /= STATE_NBR THEN¶        PUT( "*** RULE TABLE INCORRECT." );¶        INT_IO.PUT(¬RULE_NBR );¶        INT_IO.PUT(¬RTBL( RU
     LE_NBR ).STATE_NBR );¶        INT_IO.PUT(¬STATE_NBR¬);¶        NEW_LINE;¶        RETURN;¶      END¬IF;¶¶      LOOP¶        SYL_LIST :=¬TAIL( SYL_LIST
     );¬¬¬-- CAN'T BE EMPTY¶        EXIT WHEN IS_EMPTY( SYL_LIST );¶        SYL := HEAD( SYL_LIST);¶        EXIT WHEN SYL.TY = DN_TERMINAL;¶        RULE :=
      D( XD_RULE, SYL);¶        EXIT WHEN RULE.TY = DN_VOID;¶        EXIT WHEN NOT DB( XD_IS_NULLABLE, RULE );¶      END¬LOOP;¶      IF IS_EMPTY( SYL_LIST
     )¬THEN¶        FOLLOW := RTBL( RULE_NBR ).FOLLOW;¶        FOLLOW_SEQ := LIST( FOLLOW );¶        FOLLOW_SAVE¬:= FOLLOW_SEQ;¶      FOLLOW_SEQ := TERM_
     LIST.UNION( FOLLOW_SEQ, LIST( D(¬XD_FOLLOW, ITEM ) )¬);¶¶        IF NOT TERM_LIST.SAME( FOLLOW_SEQ, FOLLOW_SAVE ) THEN¶¬LIST( FOLLOW, FOLLOW_SEQ );¶¬R
0040 TBL( RULE_NBR ).REQ_CHECK :=¬TRUE;¶¬MORE_CLOSURE_PASSES¬:= TRUE;¶¬END IF;¶      END IF;¶    END¬TRANS_CLOSE_CLOSURE_ONE_ITEM;¶¶    BEGIN¶    FOR
     ¬STATE_NBR¬IN STBL'RANGE LOOP¶        IF STBL( STATE_NBR ).CHANGED THEN¶¬STBL( STATE_NBR ).CHANGED := FALSE;¶¬STATE := STBL( STATE_NBR ).STATE;¶¬ITEM_
     SEQ := LIST( STATE );¶¬IF STBL( STATE_NBR ).CLOSURE THEN¶¬  PUT ( "CL ");¶¬  INT_IO.PUT( STATE_NBR, 1 );¶¬  NEW_LINE;¶¬  ITEM_SUBSEQ := ITEM_SEQ;¶¬  W
     HILE NOT IS_EMPTY( ITEM_SUBSEQ) LOOP¶¶MAKE_RTBL_ONE_ITEM:¶¬  DECLARE¶¬    ITEM¬¬: TREE¬:= HEAD( ITEM_SUBSEQ );¶¬    RULE¬¬: TREE;¶¬    RULE_NBR¬: INTE
     GER;¶¬  BEGIN¶¬    IF DI( XD_SYL_NBR, ITEM )¬= 0 THEN¬¬¬-- CLOSURE ITEM¶¬      RULE := D( XD_RULE, D( XD_ALTERNATIVE, ITEM¬) );¶¬      RULE_NBR := DI(
      XD_RULE_NBR, D( XD_RULEINFO, RULE ) );¶¬      DECLARE¶¬        RTBL_I: RTBL_TYPE RENAMES RTBL(¬RULE_NBR );¶¬      BEGIN¶¬        IF RTBL_I.STATE_NBR
     /= STATE_NBR THEN¬¬¬-- HAVE NOT ALREADY¬SEEN A CLOSURE ITEM¬FOR THIS RULE¶¬¬RTBL_I.STATE_NBR :=¬STATE_NBR;¶¬¬RTBL_I.REQ_CHECK :=¬FALSE;¶¬¬RTBL_I.FOLLOW
      := D(¬XD_FOLLOW, ITEM);¶¬        END IF;¶¬      END;¶¬    END IF;¶¬  END MAKE_RTBL_ONE_ITEM;¶¶¬  ITEM_SUBSEQ := TAIL( ITEM_SUBSEQ );¶¬END LOOP;¶¬¬¬¬-
     - CHECK ALL ITEMS ONCE¶¬MORE_CLOSURE_PASSES¬:= FALSE;¶¬ITEM_SUBSEQ := ITEM_SEQ;¶¬WHILE NOT¬IS_EMPTY(¬ITEM_SUBSEQ ) LOOP¶¬  TRANS_CLOSE_CLOSURE_ONE_ITE
     M( STATE_NBR, HEAD( ITEM_SUBSEQ ) );¶¬  ITEM_SUBSEQ := TAIL( ITEM_SUBSEQ );¶¬END LOOP;¶¬¬¬¬-- NOW CHECK ITEMS THAT HAVE BEEN CHANGED¶¬WHILE MORE_CLOSU

0050 RE_PASSES LOOP¶¬  MORE_CLOSURE_PASSES := FALSE;¶¬  ITEM_SUBSEQ := ITEM_SEQ;¶¬  WHILE NOT IS_EMPTY( ITEM_SUBSEQ) LOOP¶¬    ITEM := HEAD( ITEM_SUBSEQ);¶
     ¬    IF DI( XD_SYL_NBR, ITEM) /= 0 THEN¶¬      ITEM_SUBSEQ := TAIL( ITEM_SUBSEQ );¶¬    ELSE¶¬      RULE := D( XD_RULE, D( XD_ALTERNATIVE, ITEM¬) );¶¬
          RULE_NBR := DI( XD_RULE_NBR, D( XD_RULEINFO, RULE ) );¶¬      IF RTBL( RULE_NBR ).REQ_CHECK THEN¶¬        RTBL( RULE_NBR ).REQ_CHECK := FALSE;¶¬
            LOOP¶¬¬TRANS_CLOSE_CLOSURE_ONE_ITEM(¬STATE_NBR, ITEM );¶¬¬ITEM_SUBSEQ := TAIL¬( ITEM_SUBSEQ );¶¬¬EXIT WHEN¬IS_EMPTY ( ITEM_SUBSEQ );¶¬¬ITEM :=
      HEAD( ITEM_SUBSEQ );¶¬¬EXIT WHEN¬D( XD_RULE, D( XD_ALTERNATIVE, ITEM ) )¬/= RULE;¶¬        END LOOP;¶¬      ELSE¶¬        LOOP¶¬¬ITEM_SUBSEQ := TAIL(
     ITEM_SUBSEQ );¶¬¬EXIT WHEN¬IS_EMPTY(¬ITEM_SUBSEQ );¶¬¬ITEM := HEAD( ITEM_SUBSEQ );¶¬¬EXIT WHEN¬D( XD_RULE, D( XD_ALTERNATIVE, ITEM)) /= RULE;¶¬
      END LOOP;¶¬      END¬IF;¶¬    END IF;¶¬  END LOOP;¶¬END LOOP;¶        END IF;¶        PUT( "GOTO " );¶        INT_IO.PUT(¬STATE_NBR, 1 );¶        NE
     W_LINE;¶        ITEM_SUBSEQ¬:= ITEM_SEQ;¶        WHILE NOT IS_EMPTY( ITEM_SUBSEQ¬) LOOP¶¶TRANS_CLOSE_GOTO_ONE_ITEM:¶¬DECLARE¶¬  ITEM¬¬: TREE¬¬:= HEAD(
      ITEM_SUBSEQ );¶¬  ALT_NBR¬¬: INTEGER;¶¬  SYL_NBR¬¬: INTEGER;¶¬  GOTO_STATE¬: TREE¬¬:= D( XD_GOTO, ITEM¬);¶¬  GOTO_ITEMSEQ¬: SEQ_TYPE;¶¬  GOTO_ITEM¬:
     TREE;¶¬  FOLLOW¬¬: TREE;¶¬  FOLLOW_SEQ¬: SEQ_TYPE;¶¬  FOLLOW_SAVE¬: SEQ_TYPE;¶¬BEGIN¶¬  IF GOTO_STATE.TY /= DN_VOID¬THEN¶¬    ALT_NBR := DI( XD_ALT_NB
0060 R, D( XD_ALTERNATIVE,¬ITEM ) );¶¬    SYL_NBR := DI( XD_SYL_NBR, ITEM ) +¬1;¶¬    GOTO_ITEMSEQ :=¬LIST( GOTO_STATE );¶¬    LOOP¶¬      GOTO_ITEM := HEA
     D( GOTO_ITEMSEQ );¶¬      EXIT WHEN DI(¬XD_ALT_NBR, D( XD_ALTERNATIVE, GOTO_ITEM ) ) = ALT_NBR¶¬¬      AND¬THEN DI( XD_SYL_NBR, GOTO_ITEM) = SYL_NBR;¶
     ¬      GOTO_ITEMSEQ := TAIL( GOTO_ITEMSEQ );¬¬-- NEVER EMPTY, BECAUSE DESIRED ITEM IS¬IN GO TO STATE¶¬    END LOOP;¶¬    FOLLOW := D( XD_FOLLOW, GOTO_
     ITEM );¶¬    FOLLOW_SAVE := LIST( FOLLOW );¶¬    FOLLOW_SEQ := TERM_LIST.UNION( FOLLOW_SAVE, LIST( D( XD_FOLLOW, ITEM ) ) );¶¬    IF NOT TERM_LIST.SAM
     E( FOLLOW_SEQ, FOLLOW_SAVE ) THEN¶¬      MORE_PASSES := TRUE;¶¬      STBL( DI( XD_STATE_NBR,¬GOTO_STATE ) ).CHANGED := TRUE;¶¬      LIST ( FOLLOW, FOL
     LOW_SEQ );¶¬    END IF;¶¬  END IF;¶¬END TRANS_CLOSE_GOTO_ONE_ITEM;¶¶¬ITEM_SUBSEQ := TAIL( ITEM_SUBSEQ );¶        END LOOP;¶      END¬IF;¶    END LOOP;
     ¶  END TRANS_CLOSE;¶¶BEGIN¶  OPEN_IDL_TREE_FILE( NOM_TEXTE & ".lar" );¶  GR_STATE_SEQ := LIST( D( XD_STATELIST, D( XD_USER_ROOT, TREE_ROOT )¬) );¶  IN
     ITIALIZE;¶  LOOP¶    MORE_PASSES := FALSE;¶    TRANS_CLOSE;¶    EXIT WHEN NOT MORE_PASSES;¶  END LOOP;¶  CLOSE_IDL_TREE_FILE;¶--|--------------------
     ----------------------------------------------------------¶END LALR_GRMR;¶
```